



Sistema de Gerenciamento de Bancos

Trabalho Prático 1

Gabriel Canuto Câmara Souza	24.2.4087
João Pedro Seabra Nogueira	25.1.4003
Lara de Andrade Pereira	25.1.4012
Luiz Felipe Bento Ferreira	24.1.4026
Maurício de Oliveira Santos Rodrigues	25.1.4020

Professor: Guillermo Camara Chavez

Ouro Preto - MG

01 de Junho de 2026

Sumário

1. Introdução	3
2. Descrição do Sistema	4
3. Decisões do Projeto	5
4. Arquitetura do Projeto	6
5. Diagrama UML	7
6. Testes Realizados	8
7. Vídeo de Apresentação	9
8. Conclusão	10
9. Anexos	11

1. Introdução

Este trabalho prático apresenta o desenvolvimento de um Sistema de Gerenciamento Bancário na linguagem C++. O objetivo principal foi aplicar de forma prática os conceitos de Programação Orientada a Objetos (POO) e utilizar as estruturas de dados fornecidas pela biblioteca STL.

O sistema simula o funcionamento básico de um banco, envolvendo o controle de contas correntes e poupança, clientes, gerentes, transações financeiras e cartões de crédito. O funcionamento ocorre por meio de uma interface interativa no terminal, e a persistência dos dados é garantida através do salvamento em arquivos de texto, evitando a perda de informações ao encerrar o programa.

2. Descrição do Sistema

O programa foi estruturado em camadas para separar a lógica de negócio da interface com o usuário. As principais classes que compõem o sistema são:

- **Pessoa:** Classe base que centraliza os dados comuns de qualquer usuário, como nome, ocupação, login e senha de acesso.
- **Cliente:** Classe derivada de Pessoa que adiciona as informações financeiras, como o saldo da conta, remuneração, tipo de conta (corrente ou poupança), histórico de transações e o vínculo com um cartão de crédito.
- **Gerente:** Classe também derivada de Pessoa que possui uma lista com os clientes associados à sua responsabilidade administrativa.
- **Cartão de Crédito:** Classe associada ao Cliente para controlar o limite disponível, o histórico de gastos e o valor acumulado da fatura atual.
- **Transação:** Registro fixo de qualquer movimentação financeira (transferência, depósito ou saque), contendo o valor, o tipo da operação e os respectivos objetos de Data e Horário.

Funcionamento dos Menus

A interação do usuário com o sistema ocorre inteiramente via terminal de texto, utilizando uma abordagem de **Menu Principal com Autenticação Sob Demanda**. Diferente de sistemas tradicionais que exigem uma tela de login logo na inicialização, este projeto permite a livre navegação pelo Menu Principal e solicita as credenciais de segurança (seja de Cliente ou de Gerente) apenas no momento exato em que uma operação restrita é acionada.

O fluxo principal do sistema foi estruturado da seguinte forma:

- **Menu Principal:** Concentra todas as grandes áreas do banco (Cadastro, Operações Bancárias, Consultas, Associação de Gerente, Rendimento da Poupança e Cartão de Crédito). É controlado por um laço de repetição que garante o funcionamento contínuo do programa até que a opção de encerramento (0) seja escolhida.
- **Submenus Temáticos:** Opções como "Operações Bancárias", "Consultas" e "Cartão de Crédito" direcionam o usuário para submenus específicos, mantendo a interface limpa e organizada.
- **Segurança e Validação Sob Demanda:** Sempre que o usuário tenta realizar uma ação que exige autorização (como fazer um saque, exibir o extrato de um cliente ou listar contas vinculadas a um gerente), o sistema invoca métodos de autenticação. O programa busca o usuário correspondente na lista em memória (carregada via arquivos CSV) e valida a senha informada antes de liberar a funcionalidade.

Essa decisão de projeto simplifica o uso do sistema, permitindo transitar entre ações de diferentes contas de forma dinâmica, sem a necessidade de realizar logoff e login globais a cada mudança de operação.

3. Decisões do Projeto

Uso da Orientação a Objetos

1. **Herança:** A herança foi aplicada para reaproveitar código de forma eficiente. A classe base **Pessoa** foi criada para concentrar atributos como nome, login e senha. As classes **Cliente** e **Gerente** herdam publicamente de **Pessoa**, o que evitou a repetição desses mesmos atributos em vários arquivos.
2. **Encapsulamento:** Todos os atributos críticos das classes foram definidos de modo privado (**private**). O acesso e a alteração desses dados ocorrem estritamente por meio de funções públicas (**getters** e **setters**). Isso garante que o saldo ou o limite de um cartão não sejam modificados de forma incorreta por outras partes do sistema.
3. **Polimorfismo:** O polimorfismo de sobreposição foi utilizado no método **exibirDados()**. Este método foi declarado na classe base **Pessoa** e reescrito em **Cliente** e **Gerente**. Dessa forma, ao chamar a função, o programa executa a versão correspondente à classe filha, exibindo os dados específicos de cada entidade.

Estruturas da STL

O uso de `std::vector` foi amplo para gerenciar as listas dinâmicas do programa, como a lista de clientes do banco e o histórico de transações de cada conta. A escolha do **vector** justifica-se pelo gerenciamento automático da memória e pela eficiência no acesso aos elementos. Para a leitura e escrita de arquivos, foram utilizadas as bibliotecas padrão de fluxo de texto (`ifstream`, `ofstream` e `stringstream`).

Persistência em Arquivos CSV

Para salvar os dados do banco de forma permanente, foi desenvolvida uma lógica de persistência utilizando dois arquivos de texto: **clientes.csv** e **gerentes.csv**.

- **Escrita:** Foi realizada a sobrecarga do operador `<<`. No momento de guardar os dados, as informações das classes são convertidas em linhas de texto, com os atributos separados por ponto e vírgula (`;`) ou barra (`|`) no caso de listas internas de transações associadas.
- **Leitura:** Sempre que o programa é iniciado, os arquivos CSV são abertos e lidos linha por linha através de `std::getline`. As informações são divididas pelos delimitadores para reconstruir os objetos na memória.

4. Arquitetura do Projeto

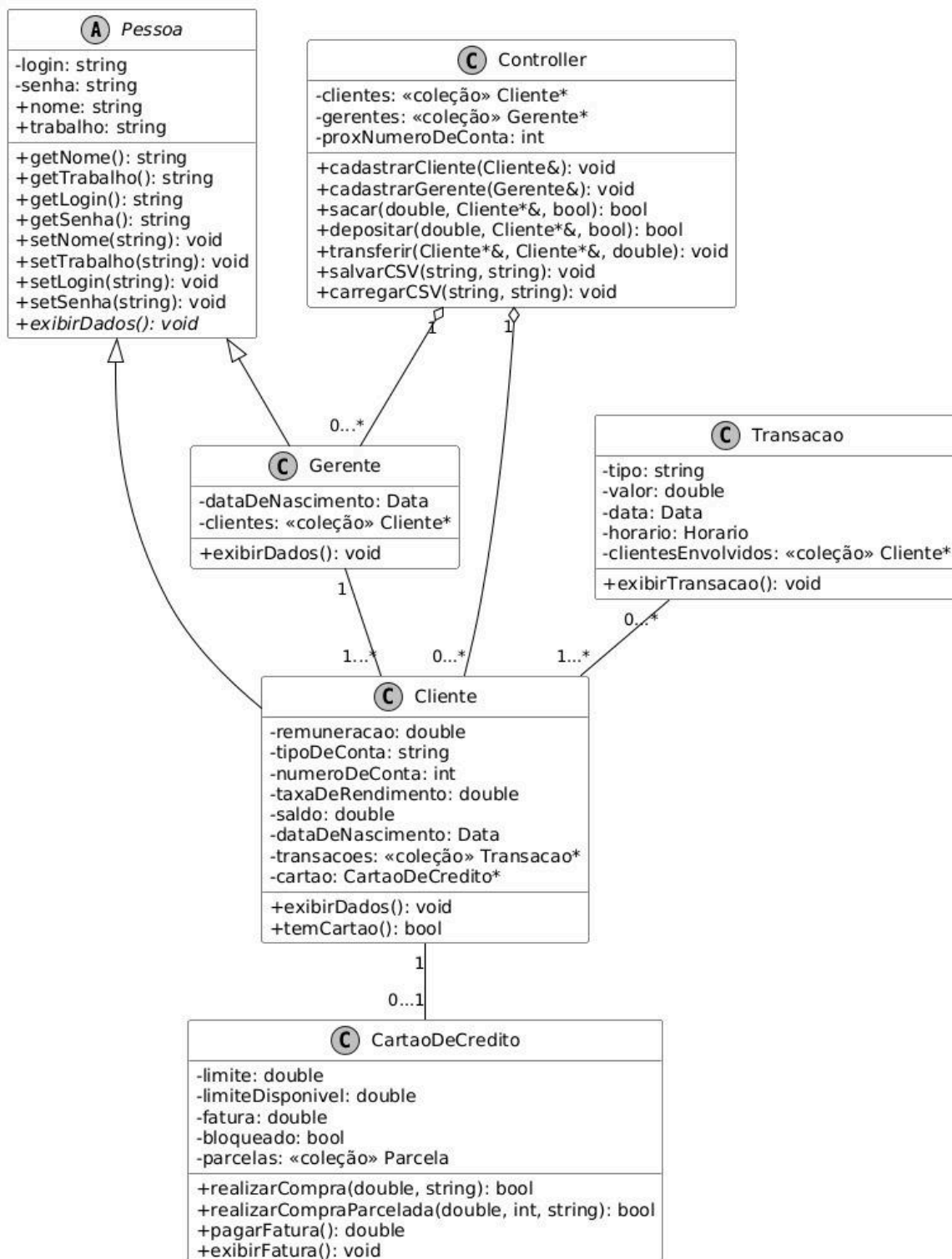
O projeto adota uma organização modular para manter o código limpo e legível. Os arquivos foram divididos entre o diretório de cabeçalhos (`imports`) e o diretório de implementações (`src`).

Divisão dos Arquivos e Responsabilidades

- **main.cpp:** Ponto de entrada do programa. O arquivo é direto e possui apenas a função de iniciar o menu principal através do comando `Menu().iniciar();`.
- **Makefile:** Script para automatizar a compilação do projeto com o compilador `g++`. Ele realiza a compilação de cada módulo diariamente, armazena os arquivos de objeto (`.o`) na pasta `build` e gera o executável final chamado `exe`. Também disponibiliza comandos como `make run` e `make clean`.
- **Pessoa, Cliente e Gerente:** Classes responsáveis pela definição dos usuários e das suas respectivas permissões no sistema.
- **CartaoDeCredito e Transacao:** Módulos que tratam da lógica financeira e das compras efetuadas.
- **Data e Horario:** Classes auxiliares utilizadas para formatar o momento exato das operações financeiras.
- **Controller:** Classe centralizadora de controle. Gerencia os vetores de ponteiros dos usuários, valida as transferências entre as contas e realiza a liberação completa de memória de todos os objetos alocados dinamicamente no seu destrutor, prevenindo vazamentos de memória (`memory leaks`).
- **Menu:** Camada responsável pela interface, exibindo as opções na tela e capturando as entradas enviadas pelo teclado.

5. Diagrama UML

A figura do **Diagrama de Classes UML do Sistema Bancário** a seguir ilustra a estrutura do sistema, demonstrando a aplicação da herança a partir da classe base **Pessoa**, bem como as relações de composição e agregação entre as entidades financeiras e a classe central de controle (Controller).



6. Testes Realizados

Cenários de Teste e Validações

Durante a fase de desenvolvimento, foram executados diversos testes para comprovar a eficácia das regras de negócio estabelecidas:

- **Geração de Contas:** Verificou-se que o sistema atribui o número das contas de forma automática e sequencial, iniciando a partir do número 1000.
- **Logins Únicos:** Testou-se a criação de novos usuários e constatou-se que o sistema bloqueia novos registros caso o login informado já exista no banco de dados.
- **Fluxo do Cartão de Crédito:** O fluxo de compras no crédito foi testado com sucesso, comprovando a redução imediata do limite disponível, o acréscimo correto no valor da fatura atual e o desconto do valor correspondente no saldo da conta corrente após a realização do pagamento da fatura.

Tratamento de Erros de Entrada

Foram implementadas validações no arquivo `Menu.cpp` para garantir o funcionamento contínuo do programa, mesmo diante de entradas incorretas:

1. **Entradas Inválidas no Menu:** Caso o menu espere um valor numérico inteiro e o usuário insira um caractere de texto, o código captura a falha, limpa o estado de erro do fluxo com `cin.clear()`, limpa o buffer com `cin.ignore()` e solicita uma nova digitação, evitando loops infinitos.
2. **Saldos e Limites Insuficientes:** Operações de saque ou transferência com valores superiores ao saldo disponível são interceptadas e bloqueadas. O mesmo comportamento foi validado para compras no crédito que excedam o limite disponível ou tentativas de uso em cartões bloqueados.

7. Vídeo de Apresentação

Para ver o sistema funcionando na prática, gravamos um vídeo demonstrando a execução das principais operações de clientes e gerentes. O link para assistir está logo abaixo:

Link do Vídeo:  ygi-rtyc-qug (2026-06-03 01:30 GMT)

8. Conclusão

O desenvolvimento deste trabalho facilitou o entendimento de como os conceitos teóricos de Orientação a Objetos funcionam na prática. O uso de herança e encapsulamento ajudou a deixar as classes bem divididas e o código mais organizado.

A parte que exigiu mais atenção no projeto foi o uso de ponteiros dentro do `std::vector<Cliente*>`. Foi necessário criar um destrutor na classe **Controller** para aplicar o comando **delete** em todos os objetos criados dinamicamente, garantindo que a memória do computador fosse limpa corretamente para evitar o acúmulo de dados desnecessários.

Como pontos de melhoria para o futuro, indica-se a substituição dos arquivos CSV por um banco de dados e a criação de uma interface gráfica com janelas no lugar do terminal de texto.

9. Anexos

Estrutura de Diretórios do Projeto

```
.
├── main.cpp
├── Makefile
├── clientes.csv          # Arquivo de persistência de Clientes
├── gerentes.csv         # Arquivo de persistência de Gerentes
├── exe                  # Programa executável gerado
├── imports/             # Diretório contendo os cabeçalhos (.h)
│   ├── Pessoa.h
│   ├── Cliente.h
│   ├── Gerente.h
│   ├── Transacao.h
│   ├── CartaoDeCredito.h
│   ├── Controller.h
│   ├── Menu.h
│   ├── Data.h
│   └── Horario.h
├── src/                 # Diretório contendo as implementações (.cpp)
│   ├── Pessoa.cpp
│   ├── Cliente.cpp
│   ├── Gerente.cpp
│   ├── Transacao.cpp
│   ├── CartaoDeCredito.cpp
│   ├── Controller.cpp
│   ├── Menu.cpp
│   ├── Data.cpp
│   └── Horario.cpp
└── build/               # Diretório dos objetos de compilação (.o)
```

Inicialização e Menu Principal

```
=====
                        SISTEMA DE GERENCIAMENTO DE BANCO
=====
1. Cadastrar
2. Operações bancárias
3. Consultas
4. Associar gerente a um cliente
5. Listar logins cadastrados
6. Cartão de crédito
0. Salvar dados e sair
=====
Escolha uma opção: █
```

Tela inicial do programa logo após ser compilado e executado pelo Makefile

Operações de Cliente e Exibição do Extrato

```
=====
                        REALIZAR DEPÓSITO
=====
Informe o login da conta: guillermo
Informe a senha: bcc221
Informe o valor: R$500

Depósito de R$500 realizado com sucesso!
=====

Pressione enter para voltar ao menu...█
```

```
=====
                        EXIBIR EXTRATO
=====
Informe o login: guillermo
Informe a senha: bcc221

-> Transação #1
Tipo: Depósito
Valor: R$500
Data: 01/06/2026
Horário: 16:29
Clientes envolvidos: -

=====

Pressione enter para voltar ao menu...█
```

Realização de um depósito e, em seguida, a exibição da tela de **Extrato**.

Fluxo do Cartão de Crédito

```
Login do gerente: lucas.mendes
Senha: senha
Limite disponível: R$2340.00
Descrição da compra: Loja
Valor: R$1200

Compra de R$1200.00 realizada com sucesso!

=====
Pressione enter para continuar...
```

```
Login do gerente: lucas.mendes
Senha: senha
=====
                        FATURA DO CARTÃO
=====
Limite total:          R$2340.00
Limite disponível: R$1140.00
Status: Ativo
-----
Compras:
  Loja — R$1200.00
-----
Total da fatura: R$1200.00
=====
=====
Pressione enter para continuar...
```

Momento da realização de uma compra no cartão de crédito e a subsequente consulta da fatura.

Lista Associados do Gerente

```
=====
                        LISTAR GERENTE
=====
Informe o login: marcos.pinto
Informe a senha: senha

- DADOS DO GERENTE -
Login: marcos.pinto
Nome: Marcos Pinto
Data de Nascimento: 03/09/1980
Trabalho: Gerente Regional
Clientes:

Cliente #1
Nome: Lucas Mendes
Trabalho: Arquiteto
Tipo de Conta: Poupança

Cliente #2
Nome: Beatriz Santos
Trabalho: Advogada
Tipo de Conta: Corrente

=====
Pressione enter para voltar ao menu...|
```

Execução da opção que lista todos os clientes vinculados a conta administrativa.

Tratamento de Erro

```
=====
                        REALIZAR SAQUE
=====
Informe o login da conta: guillermo
Informe a senha: bcc221
Informe o valor: R$20000

Erro! Saldo insuficiente!
=====

Pressione enter para voltar ao menu...█
```

```
=====
                        SISTEMA DE GERENCIAMENTO DE BANCO
=====
1. Cadastrar
2. Operações bancárias
3. Consultas
4. Associar gerente a um cliente
5. Listar logins cadastrados
6. Cartão de crédito
0. Salvar dados e sair
=====
Escolha uma opção: a

Opção inválida! Digite novamente: b

Opção inválida! Digite novamente: █
```

Captura simulando uma falha proposital, com uma tentativa de realizar um saque com valor superior ao saldo disponível, e a digitação de uma letra em um menu que espera um número inteiro.